



Tema 2: Introducción al ML, DL y LLMs – Clase 4

Contenidos

- ¿Qué es NLP y por qué es difícil?
- De letras a números - Tokenización
- Embeddings – significado en vectores
- Modelos secuenciales – RNN y LSTM
- La revolución de la atención
- El Transformer y sus variantes
- Demo en vivo



¿Qué es NLP y por qué es difícil?



NLP en una frase

NLP es la rama de la IA que estudia cómo las máquinas pueden **entender, generar y manipular** el lenguaje humano en forma de texto o voz.

Tres familias de tareas:

Comprensión

Clasificación, análisis de sentimiento, extracción de entidades, búsqueda.

Generación

Traducción, resumen, redacción, chatbots.

Conversión

Texto ↔ voz (TTS y STT), texto ↔ imagen.

 Todo lo que hace ChatGPT está construido sobre décadas de investigación en NLP.



Por qué el lenguaje humano es difícil para una máquina

Ambigüedad léxica

"Banco" puede ser una entidad financiera o un asiento.

Ambigüedad sintáctica

"Vi al hombre con el telescopio" — ¿quién tiene el telescopio?

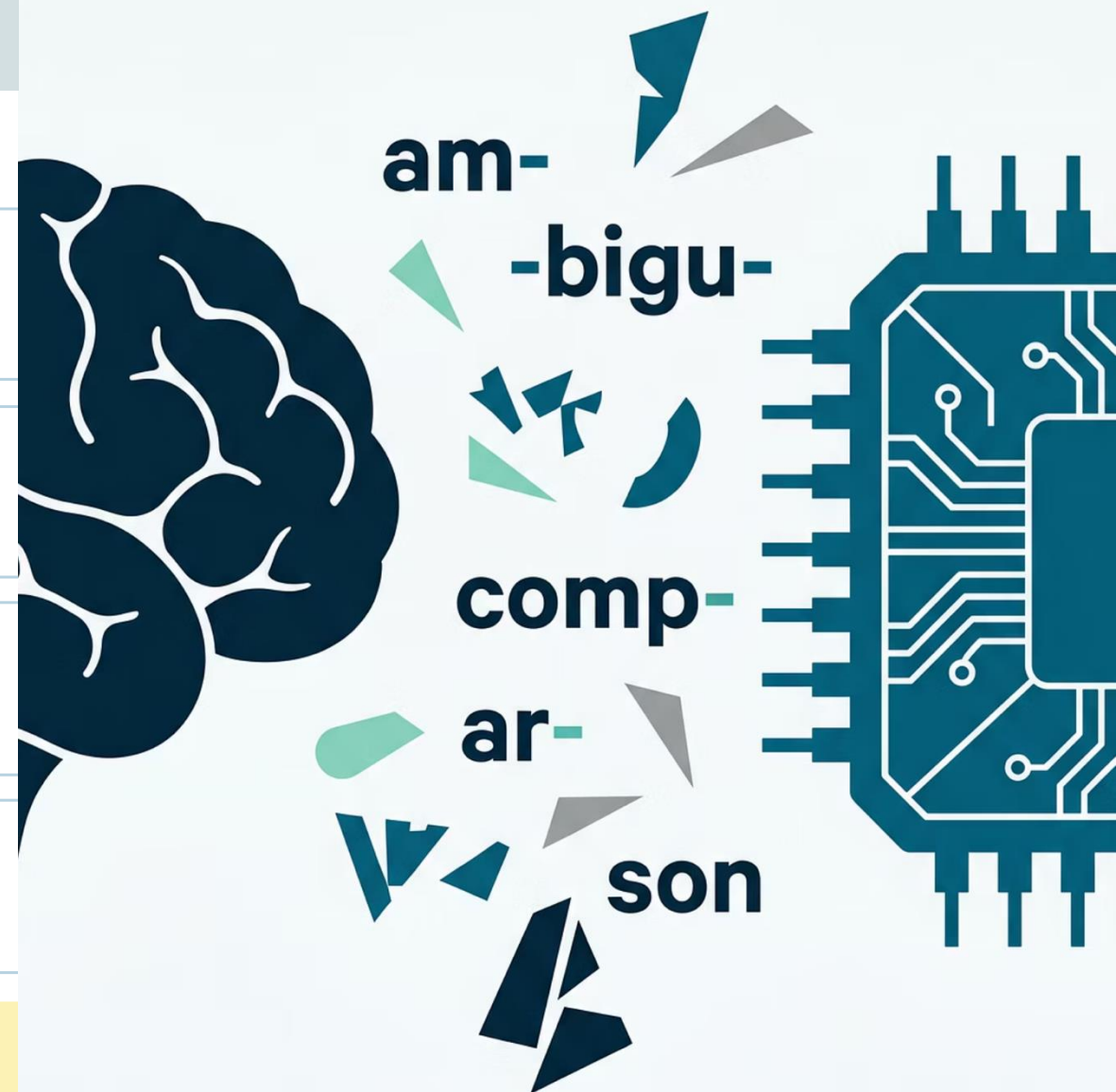
Contexto

"Vaya frío" puede ser literal o sarcástico según la entonación.

Variabilidad

La misma idea se puede expresar de mil formas distintas.

⚠ Una máquina sin contexto y sin conocimiento del mundo está perdida.



Probemos la amigüedad

Imaginad estas tres frases. ¿Qué significan exactamente?

Frase 1

"El profesor le dijo al alumno que el examen estaba mal."

→ ¿el examen de quién?

Frase 2

"El padre prohibió a su hija comer en la calle porque estaba sucia."

→ ¿quién estaba sucia?

Frase 3

"He estado leyendo a Cervantes toda la mañana."

→ ¿leyendo qué de Cervantes?



De letras a números - Tokenización



El problema fundamental

i Las redes neuronales operan con números, no con letras.

Necesitamos una función, una transformación, que haga lo siguiente:



Dos sub-problemas a resolver:

1

¿Qué unidades elegimos?

¿Caracteres? ¿Palabras? ¿Trozos de palabra?

2

¿Cómo las convertimos en números?



Métodos clásicos – One-hot y Bag-of-words

One-hot encoding

- Cada palabra del vocabulario = un vector con un único 1 y miles de 0s.
- "gato" = $[0, \dots, 0, 1, 0, \dots, 0]$ (donde el 1 está en la posición de "gato").
- **Problema:** vectores enormes, "gato" y "felino" son igual de lejanos que "gato" y "patata".

Bag-of-Words (BoW)

- Cuenta cuántas veces aparece cada palabra en un documento.
- Útil para clasificación simple (spam, sentimiento).
- **Problema:** ignora el orden – "perro muerde hombre" = "hombre muerde perro".



Funcionaba en 2010. Hoy nos parece prehistórico.



Tokenización moderna – Byte Pair Encoding (BPE)

Byte-Pair Encoding (BPE) – la tokenización que usan GPT, Claude, Llama...

→ **No tokeniza por palabras enteras**

Vocabulario infinito si usamos palabras; secuencias infinitas si usamos caracteres.

→ **Subwords: fragmentos frecuentes**

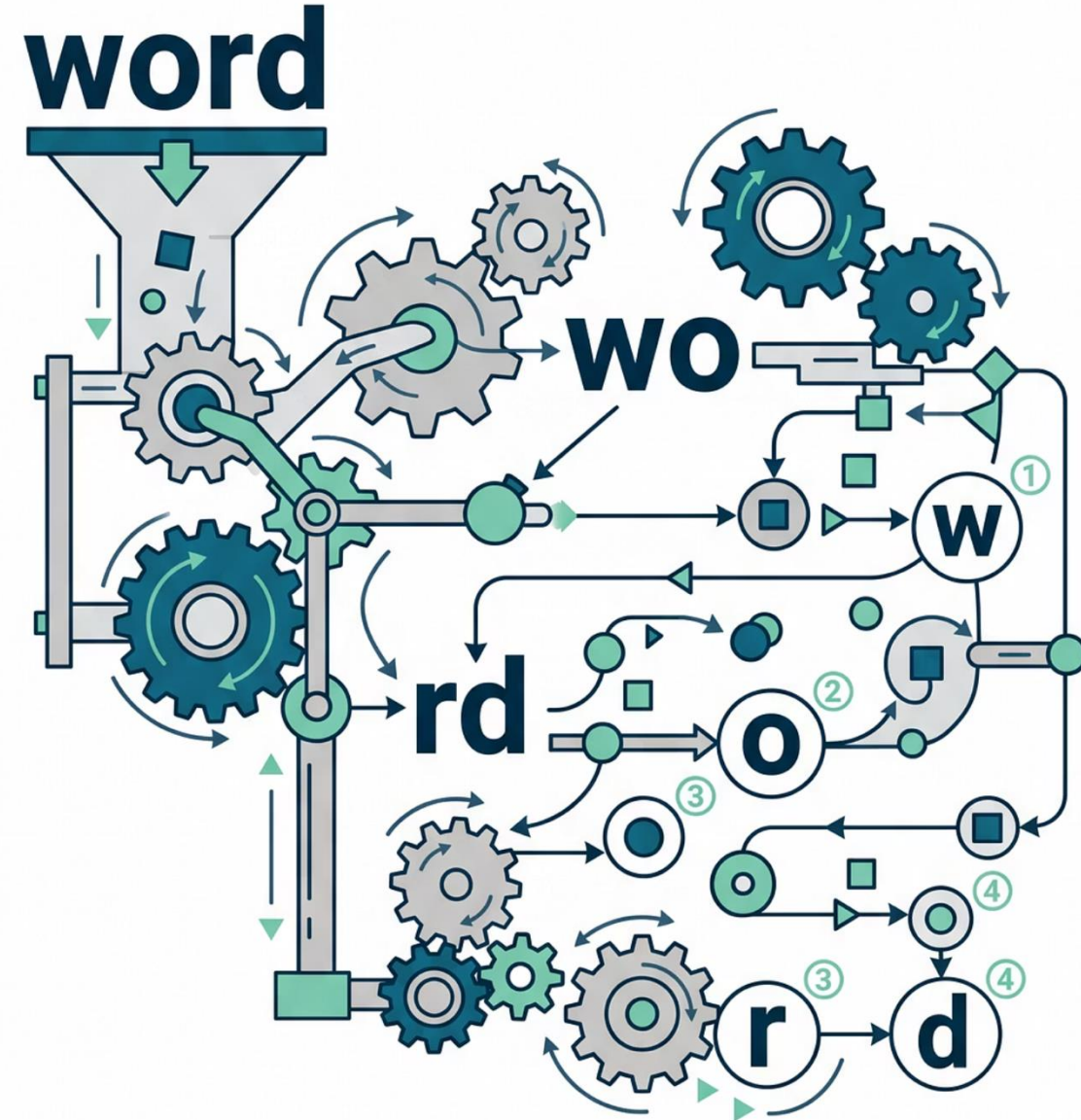
Palabras comunes → un solo token: "casa", "el", "the".

→ **Palabras raras → varios tokens**

"tokenización" → ["token", "iz", "ación"].



Cada token tiene un ID numérico en un vocabulario fijo (~50k tokens en GPT-4, ~100k en GPT-4o). Resultado: cualquier texto del mundo se representa con secuencias cortas de enteros.



Mini-demo – Tokenizar con tiktoken

```
import tiktoken  
enc = tiktoken.encoding_for_model("gpt-4")
```

```
enc.encode("Hola mundo, soy un LLM.")  
# → [39, 8083, 11, 17916, 220, 11, 2641, 720, 21024, 13]
```

Observaciones clave:

6 palabras → 10 tokens

El español "trocea" más que el inglés.

"Hola" es un token entero

"LLM" se trocea porque es una palabra rara en el vocabulario.

Coste en tokens

El coste de una llamada a un LLM se mide en tokens, no en palabras.



Embeddings – Significado en vectores



¿Qué es un embedding?

Un **embedding** es una representación densa y continua de un objeto en forma de vector.



Dimensiones con significado

Cada token se mapea a un vector de 100–4096 dimensiones que captura propiedades semánticas y sintácticas.



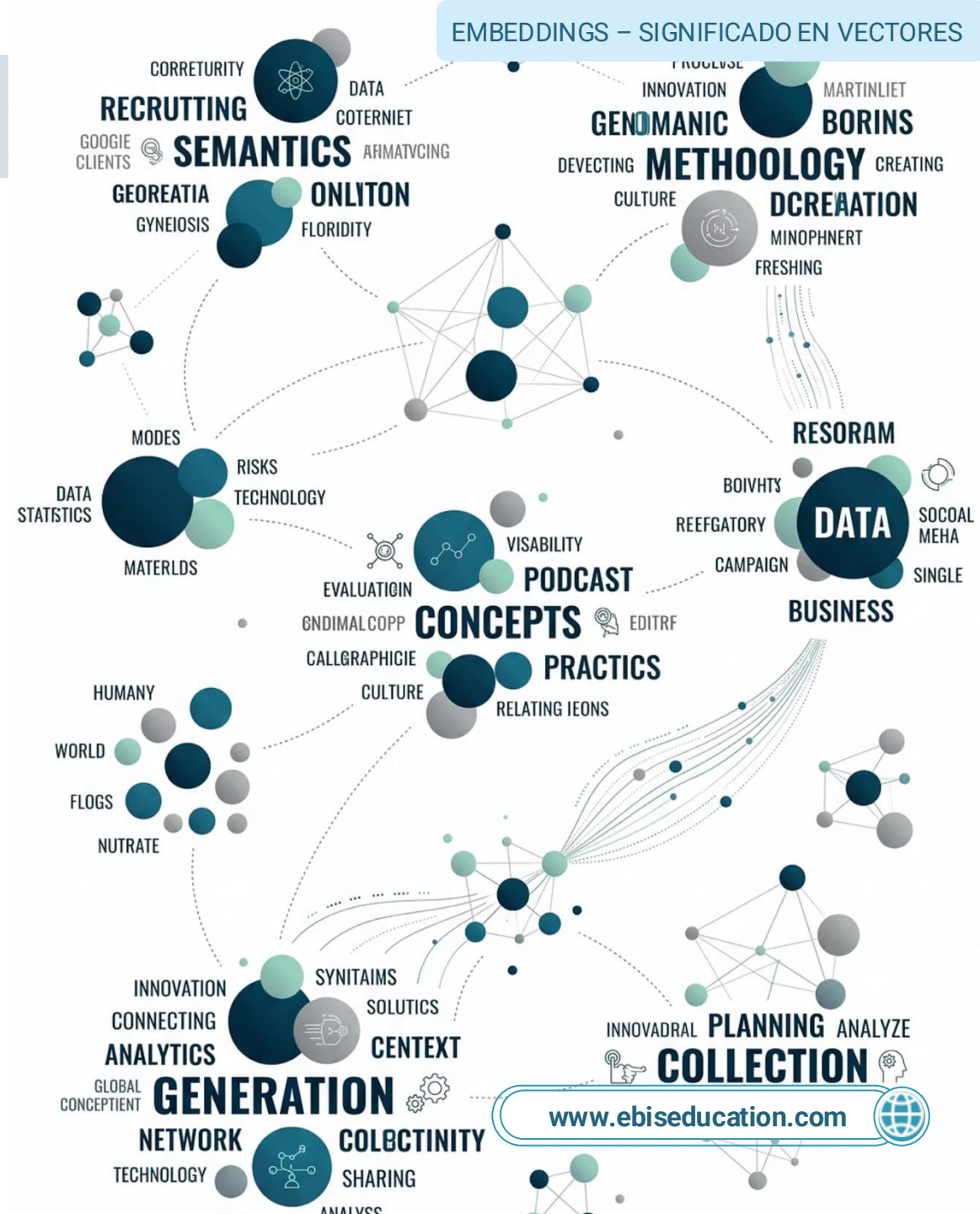
Similitud geométrica

Palabras similares → vectores cercanos en el espacio.



Significado como geometría

Los embeddings convierten "significado" en distancias y ángulos medibles.



Word2Vec y la magia de las analogías

Word2Vec (Mikolov, Google, 2013) – el primer algoritmo que reveló la aritmética vectorial del significado.

```
vector("rey") - vector("hombre") + vector("mujer") ≈ vector("reina")
```

```
vector("París") - vector("Francia") + vector("Italia") ≈ vector("Roma")
```

→ ¿Cómo lo aprende? Entrenando una red para predecir palabras vecinas en millones de frases. Las palabras que aparecen en contextos similares acaban con embeddings similares.

→ ¿Cómo funciona? Lo que nos interesa son las conexiones de la red anterior, no el output. Queremos lograr que la red aprenda regularidades sintácticas y semánticas sin que se las hayamos enseñado explícitamente.

✓ Es el momento en que la comunidad de NLP dice: "esto funciona".



Espacios semánticos - ¿Cómo se ven?

Aunque los embeddings tienen 100–4096 dimensiones, podemos proyectarlos a 2D con técnicas como **PCA** o **t-SNE** para visualizarlos.



Clusters semánticos

Animales agrupados, frutas agrupadas, países agrupados.



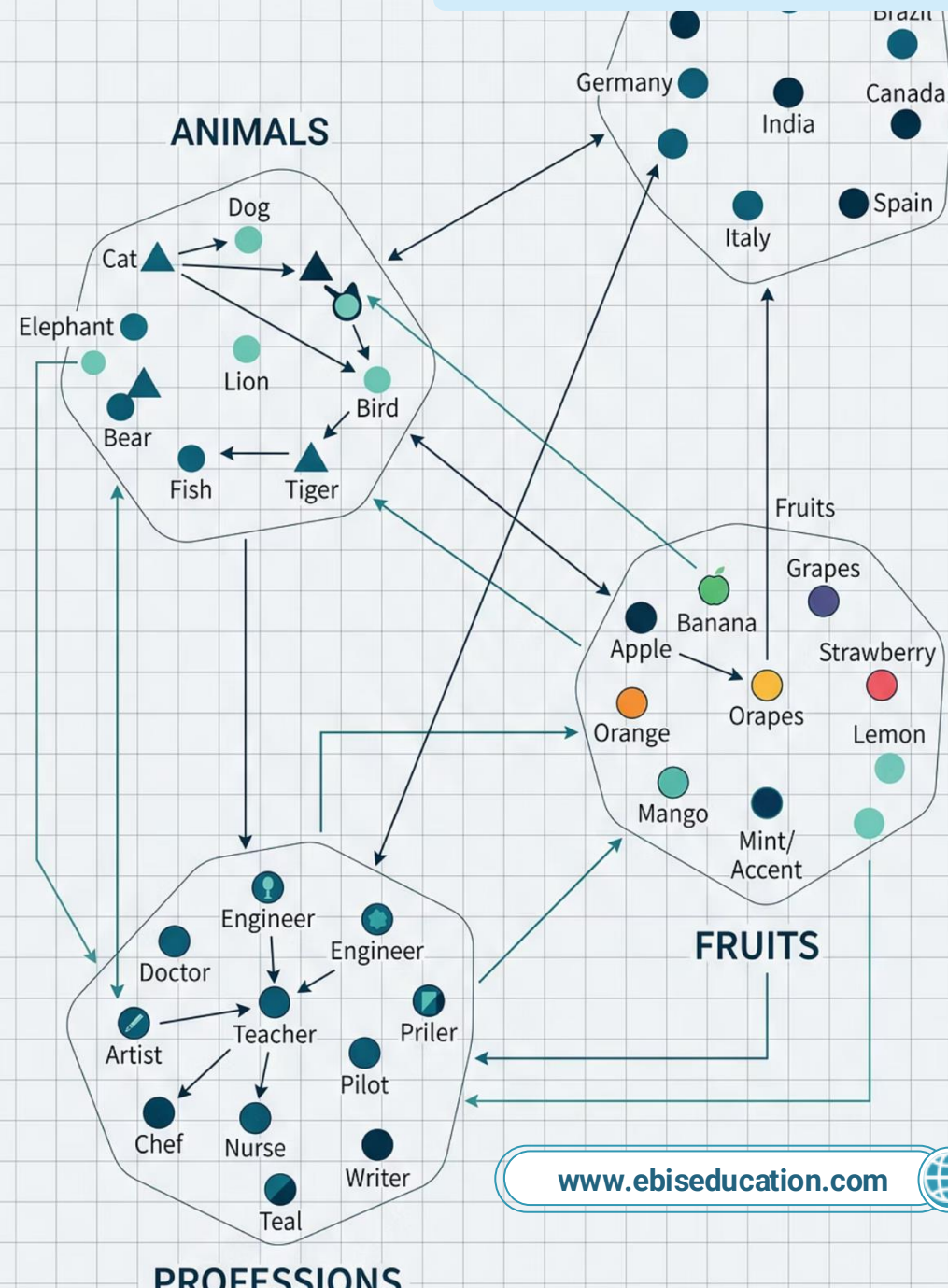
Direcciones semánticas

La dirección:
"masculino → femenino"
es consistente;
la dirección:
"presente → pasado" también.



Distancias coseno

Métrica habitual de similitud entre embeddings.



Por qué los embeddings importan tanto

Los embeddings son la base oculta de casi todo lo que vais a ver en el resto del Máster:



Búsqueda semántica

Encontrar lo similar, no lo idéntico.



RAG

Recuperar contexto relevante para alimentar a un LLM.



Clasificación moderna

Entrenar clasificadores sobre embeddings de frases.



LLMs

La primera capa de un GPT es una tabla de embeddings de tokens.



Si entendéis embeddings, entendéis cómo "piensan" los modelos de lenguaje.



Modelos secuenciales – RNN y LSTM



El reto de las secuencias

El texto no es un conjunto de palabras: es una **secuencia**.

El orden importa

"El perro mordió al hombre" $\neq$ "El hombre mordió al perro"

Modelos clásicos no sirven

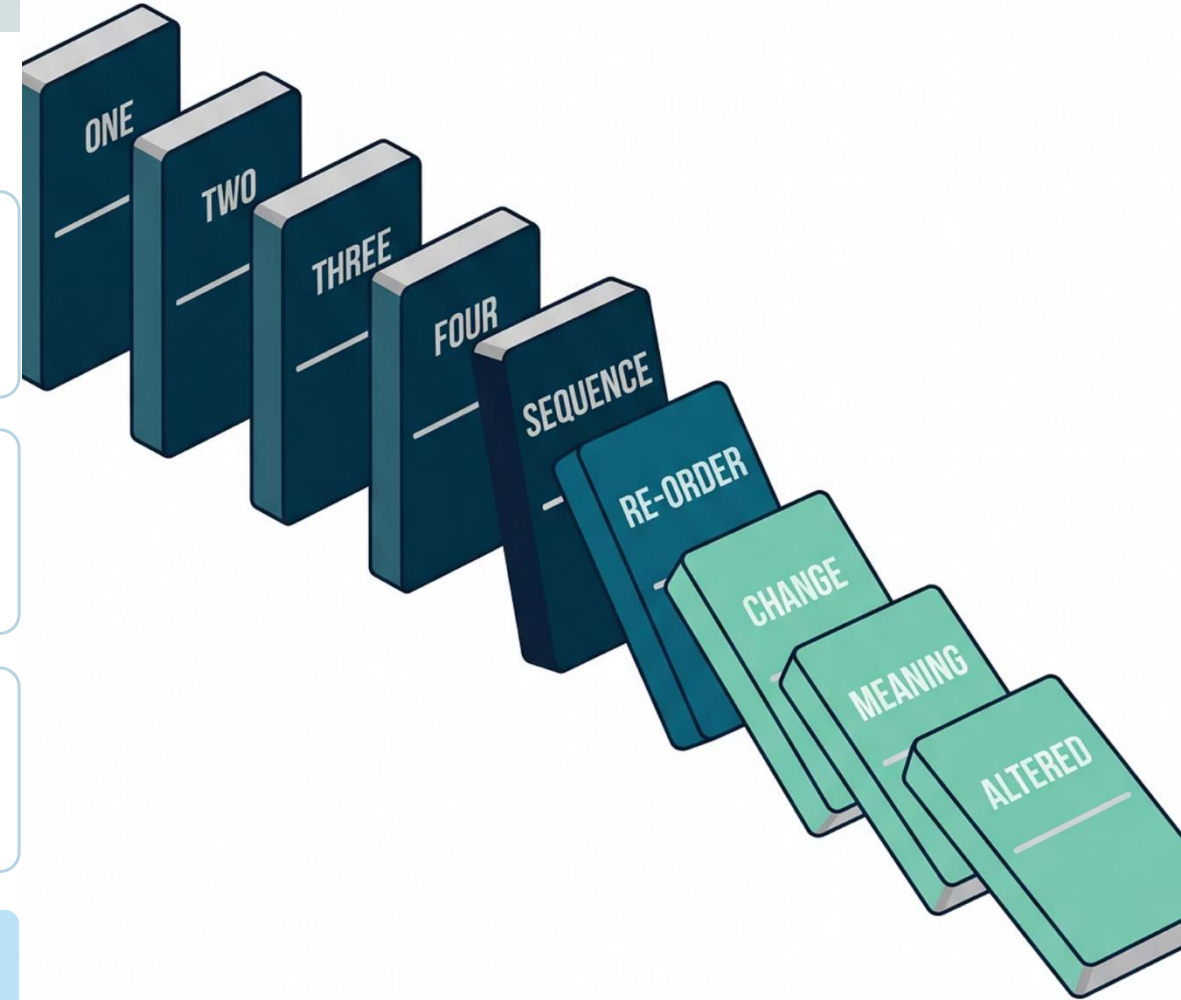
Regresión, árboles... procesan entradas independientes, sin memoria del orden.

Necesitamos memoria

Un modelo que respete el orden y recuerde lo que ya ha visto.

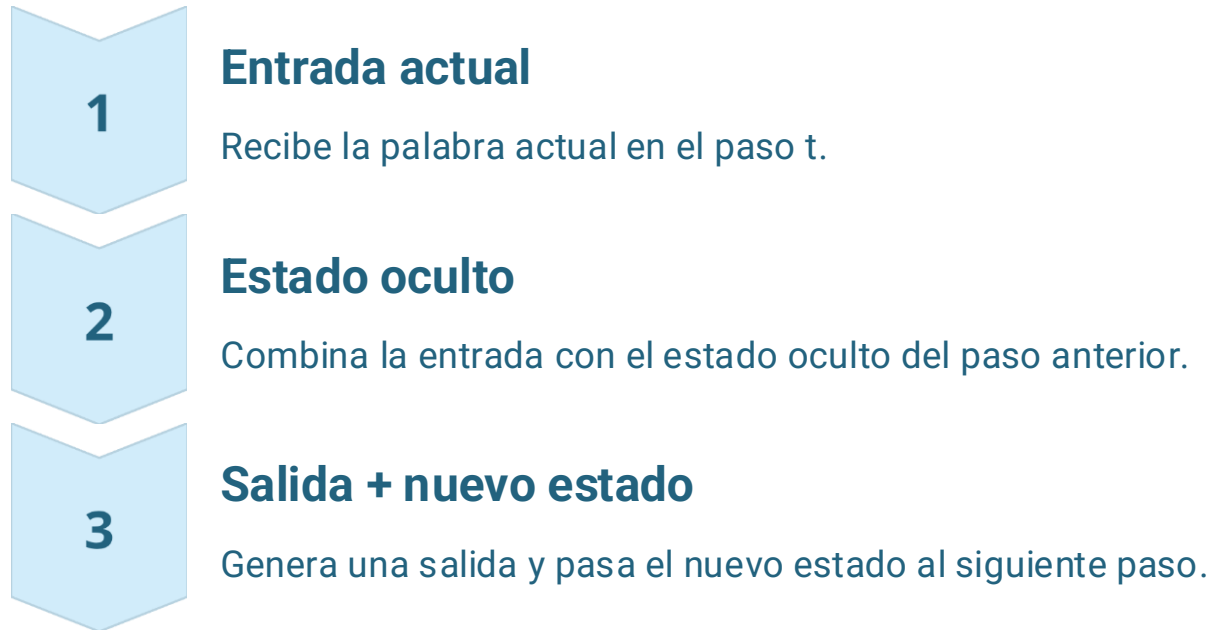


Las redes neuronales recurrentes (RNN) fueron la primera respuesta seria a este reto.

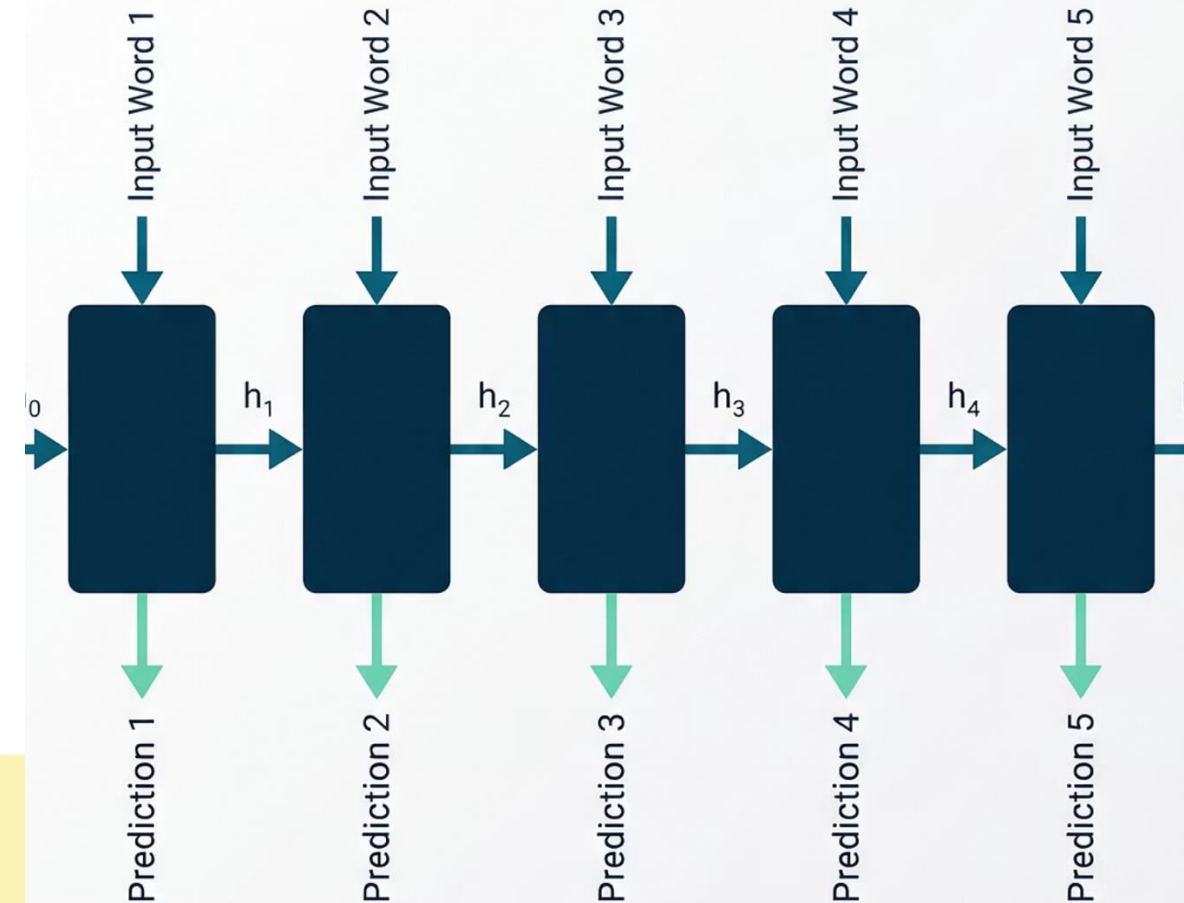


RNN – La idea de memoria interna

Una RNN procesa una secuencia **paso a paso**:

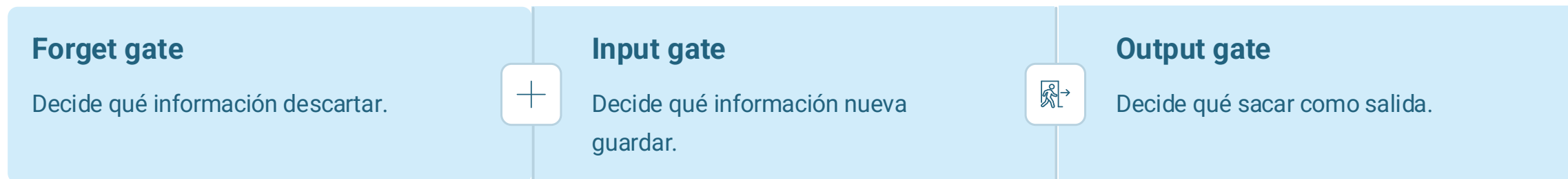


Problema clave: el gradiente se desvanece. En secuencias largas, el modelo "olvida" el principio cuando llega al final. En la práctica, las RNN no manejan bien contextos de más de 20–30 palabras.



LSTM – Memoria mejrada, pero...

LSTM (Hochreiter & Schmidhuber, 1997) introduce **gates** para gestionar qué recordar y qué olvidar.



Mejora notablemente la memoria a largo plazo. Pero...

→ Sigue siendo secuencial

No se puede paralelizar el entrenamiento (cada paso depende del anterior).

→ Contexto largo aún limitado

Manejar 1000 palabras es difícil; manejar 100.000 (lo que hace GPT-4 hoy) es imposible.

→ Entrenamiento lento

Un modelo grande puede tardar semanas.



Llegado el 2017, la comunidad ya sabía: necesitamos otra cosa.



La revolución de la atención



Intuición – “Attention is all you need”

“El gato que dormía en el sofá rojo era de mi hermana.”

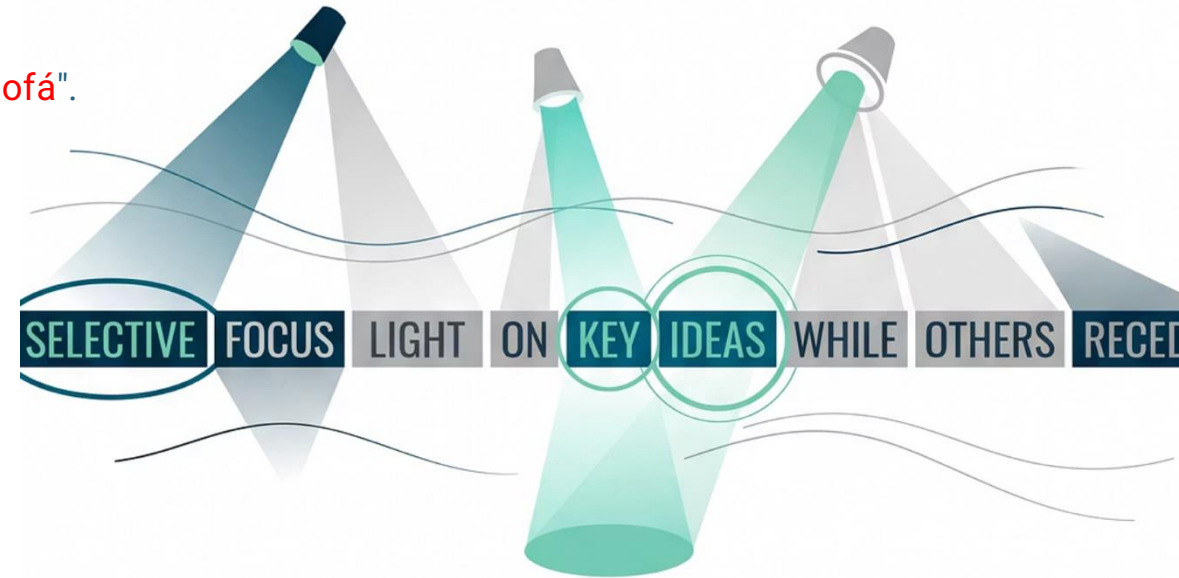
Cuando lees esta frase, al llegar a “hermana”, tu atención salta a “gato” — no a “sofá”.

Atención selectiva

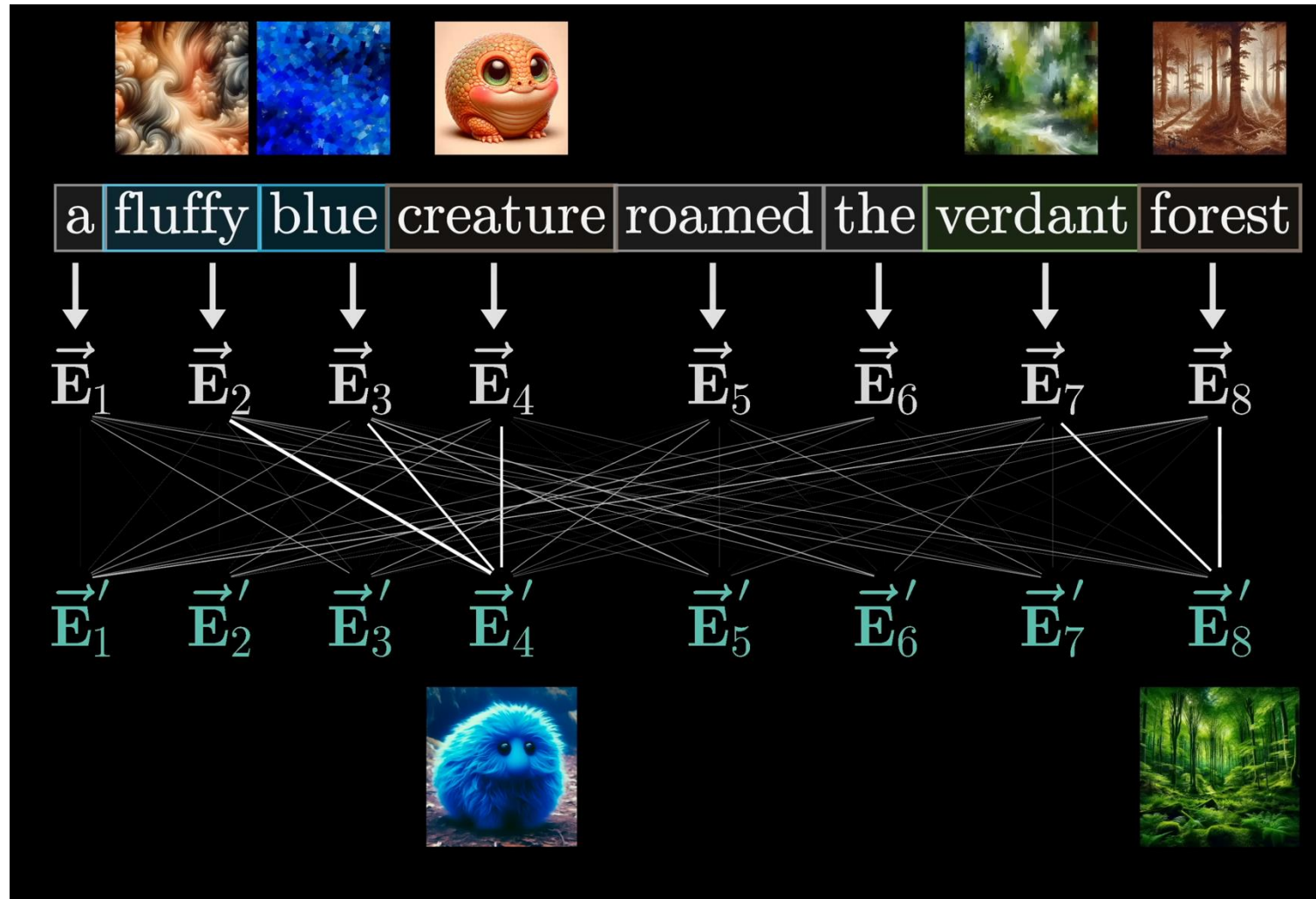
Vuestro mecanismo de atención permite mirar selectivamente otras partes de la secuencia, sin importar lo lejos que estén.

En paralelo

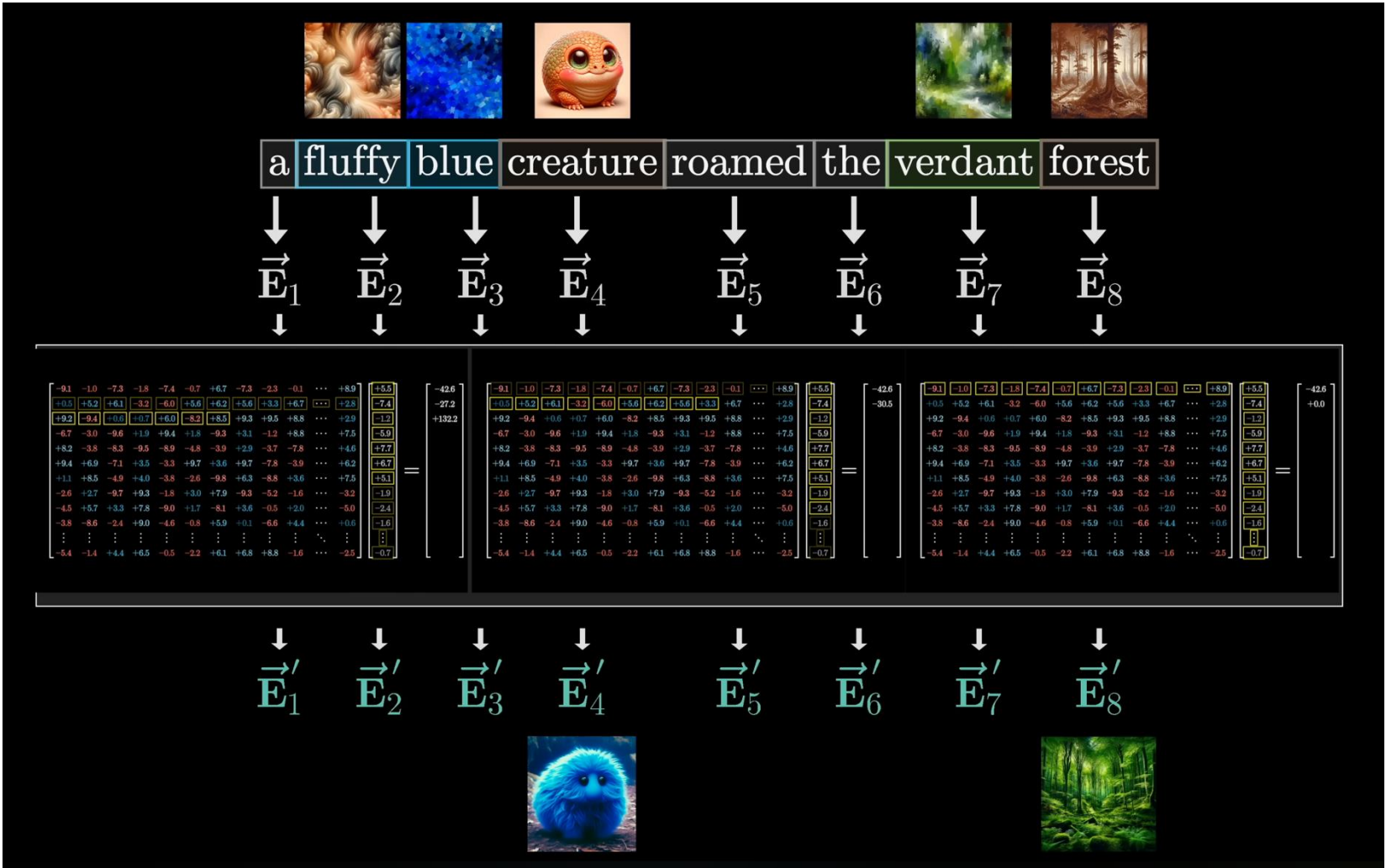
Y lo más importante: lo hacéis en paralelo, no paso a paso como lo harían las RNN.



¿Qué es lo que buscamos?



¿Qué es lo que buscamos?



Q, K, V – La analogía de la biblioteca

Imagina que buscas un libro en una biblioteca:



Query (Q)

"Libros sobre IA" – lo que estás buscando.



Key (K)

El título, autor y etiquetas visibles de cada libro.

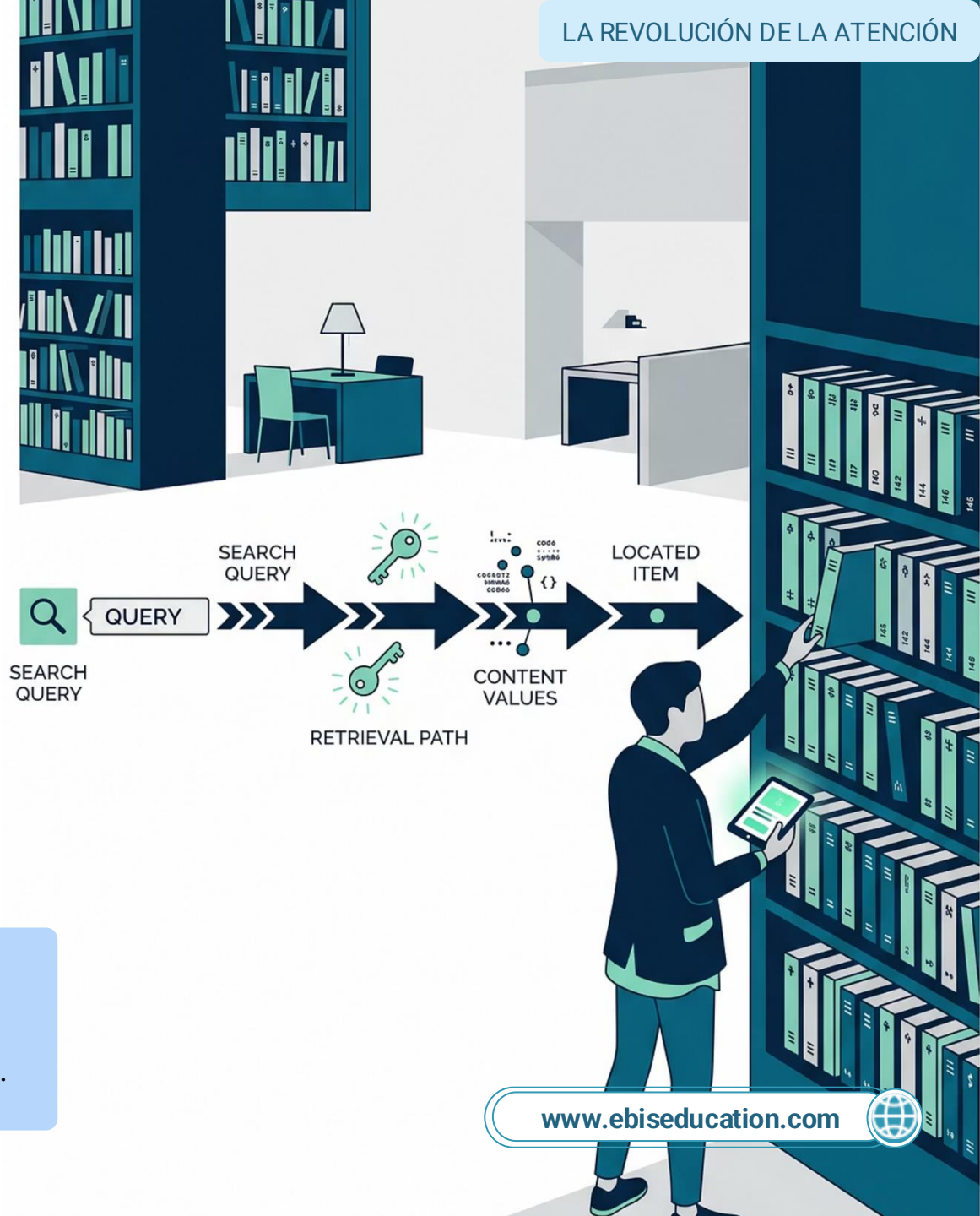


Value (V)

El contenido completo del libro.



Tú comparas tu Q con cada K, encuentras los más parecidos, y recuperas sus V. Eso es exactamente lo que hace el mecanismo de atención – pero con vectores y en paralelo, para cada palabra a la vez.



Self-attention – Cada token mira a todos

En **self-attention**, los Q, K y V vienen todos de la misma secuencia:

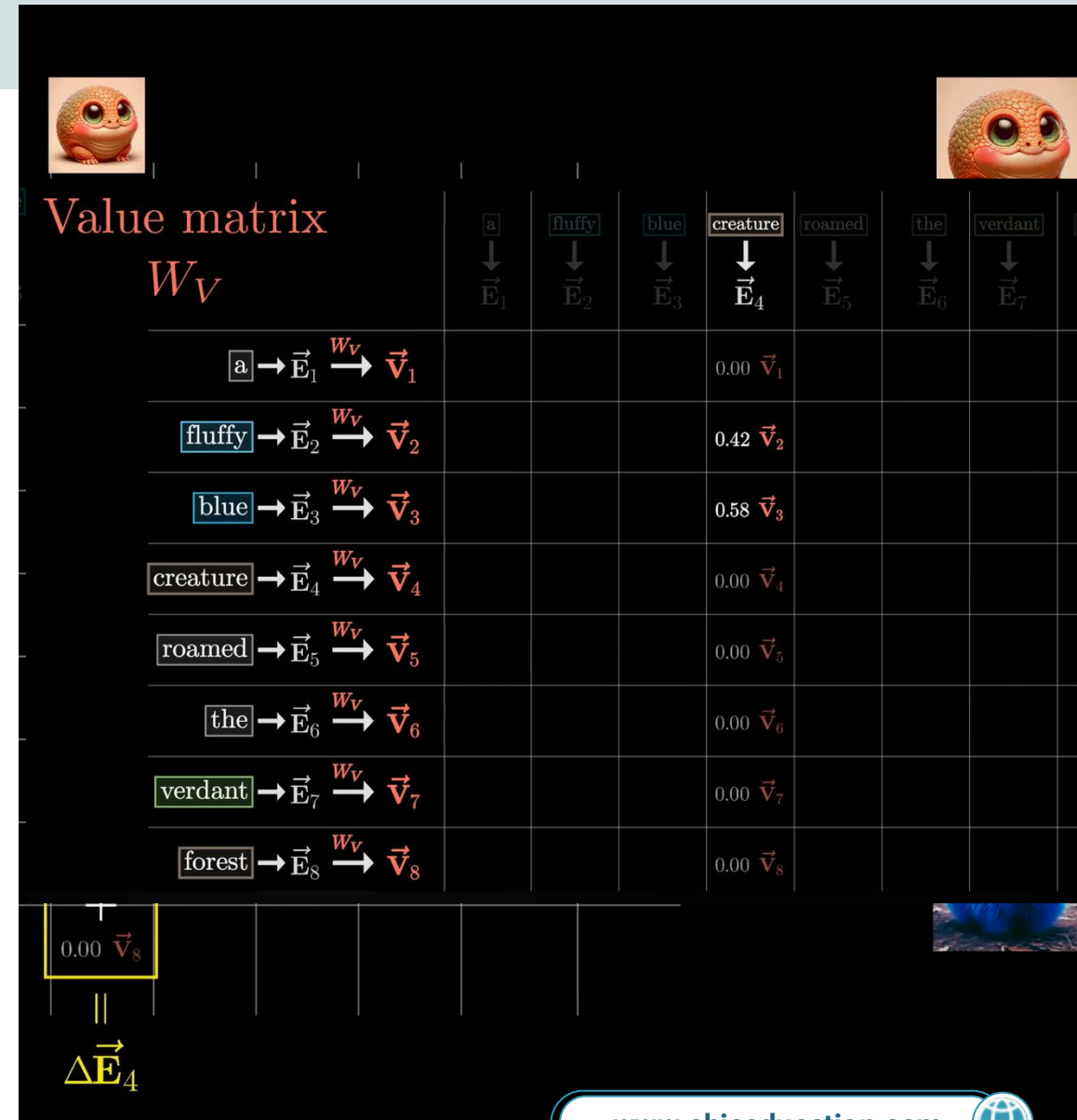
- 1 Cada token genera su propio Q, K y V**
Con matrices entrenadas específicas para cada rol.
- 2 Cada token calcula su similitud con todos los demás**
Incluido él mismo – formando una matriz de atención completa.
- 3 El resultado es una suma ponderada de los V**
De todos los tokens, según los pesos de atención calculados.

Captura dependencias largas

La palabra 1 puede mirar directamente a la 1000.

Paralelizable

Todas las palabras se calculan a la vez en una GPU.



Multi-head attention – Varios puntos de vista

En lugar de una sola operación de atención, los Transformers usan **varias en paralelo**:



Cada "head" aprende relaciones distintas

Sintáctica, semántica, posicional... cada cabeza se especializa en un tipo de relación.



Concatenación final

Los resultados de todas las heads se concatenan al final para obtener la representación completa.

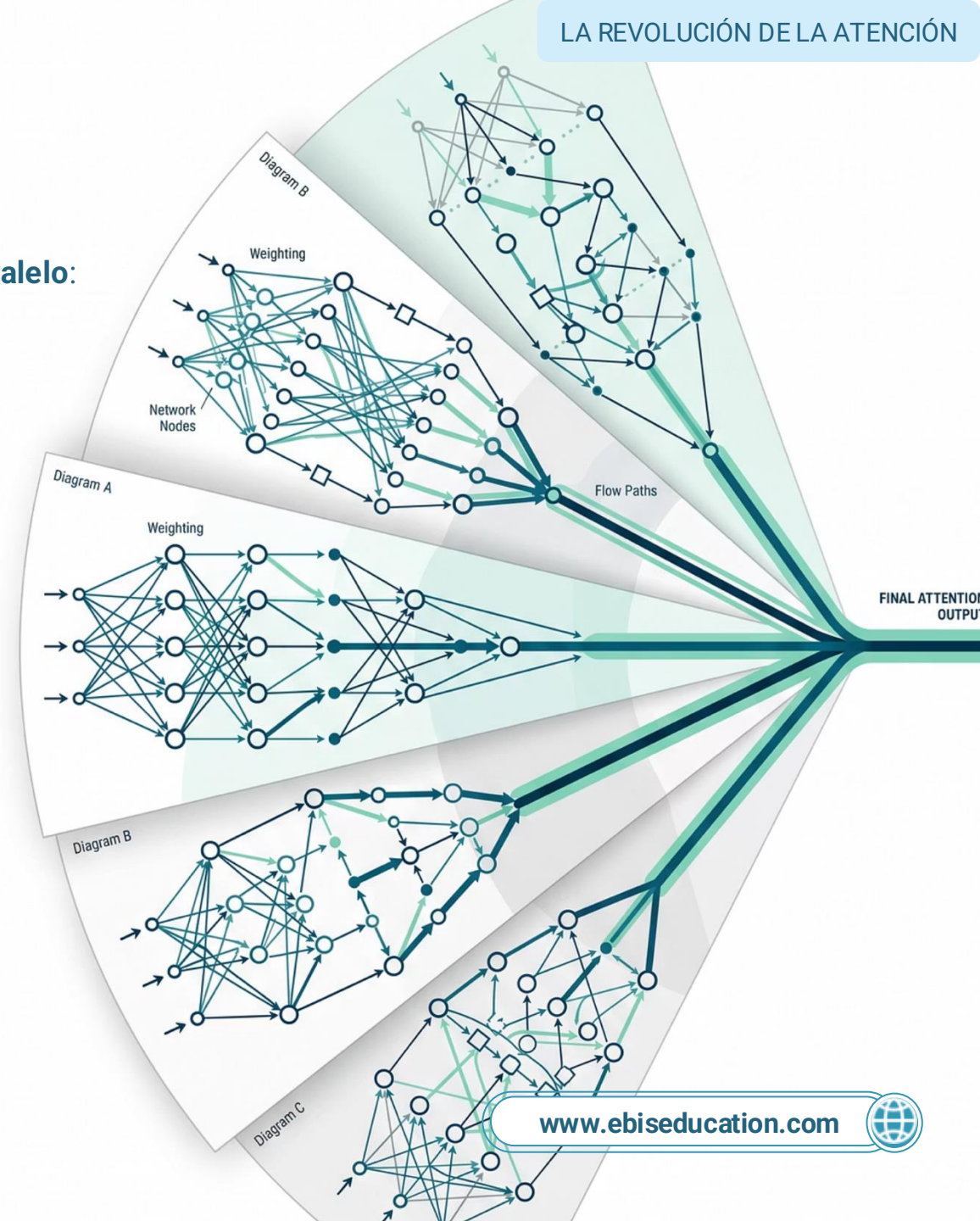


Escala en GPT-4

Aproximadamente 96 capas de Transformer, cada una con ~96 heads.



Pensad en multi-head como en varios lectores leyendo la misma frase, cada uno fijándose en algo distinto, y combinando sus interpretaciones al final.



El transformer y sus variantes



Arquitectura general - “Attention is all you need”

El **Transformer** (Google, 2017) introduce dos partes:

Encoder

Procesa la entrada completa. Cada bloque: **self-attention + feed-forward**.

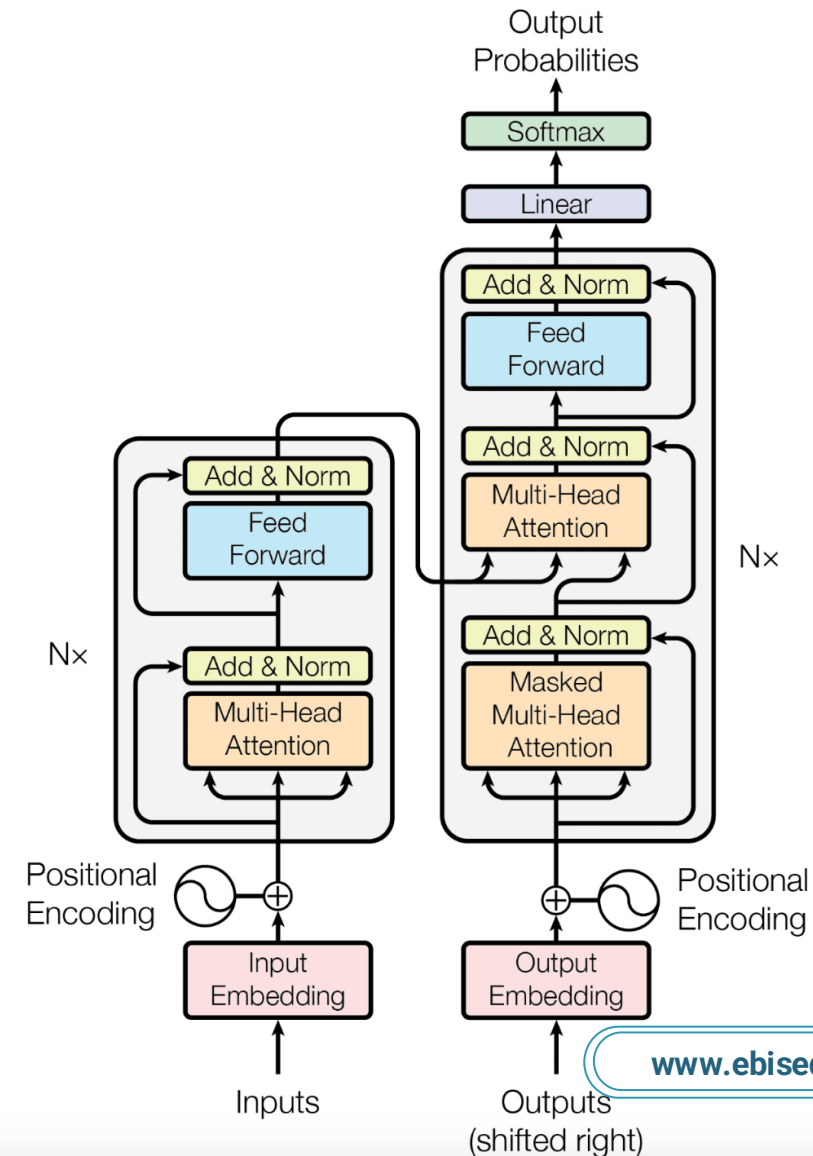
Decoder

Genera la salida token a token. Cada bloque: **masked self-attention + cross-attention + feed-forward**.

Originalmente diseñado para traducción (texto en inglés → texto en alemán).



Lo que cambió todo: a partir de aquí, las arquitecturas se simplificaron quedándose con solo encoder o solo decoder, según la tarea.



BERT – Solo encoder para comprender

Arquitectura

Solo la mitad **encoder** del Transformer. Bidireccional – ve toda la secuencia a la vez, no en orden.

Pre-entrenamiento

Masked language modeling: tapa palabras al azar y aprende a adivinarlas.

Genial para tareas de **comprensión**:

Clasificación

Sentimiento, spam, intent.

Extracción de entidades

Nombres, fechas, organizaciones.

Question Answering

Sobre un texto dado.

Búsqueda y ranking

Recuperación de información.

 No genera texto largo. Su fuerte es **entender**.



GPT – Solo decoder, para generar

Arquitectura

Solo la mitad **decoder** del Transformer. Autoregresivo – predice el siguiente token, uno a uno.

Masking causal

Cada token solo puede mirar a los anteriores, no a los futuros.

Genial para **generar**:

Conversación

Chatbots y asistentes.

Redacción

Resumen, traducción.

Código

Generación de código.

Razonamiento

Paso a paso.



Esta es la arquitectura que dominará la siguiente sesión: en T2-S5 construiréis un mini-GPT desde cero.



Comparativa de arquitecturas

Modelo	Arquitectura	Fuerte en	Ejemplos prácticos
BERT-like	Solo encoder, bidireccional	Comprensión	Clasificación, embeddings, búsqueda, NER
GPT-like	Solo decoder, autoregresivo	Generación	Chat, redacción, código, asistentes
T5-like	Encoder + Decoder	Transformación	Traducción, resumen, "text-to-text"

 Hoy la balanza se ha inclinado hacia **decoder-only**: los LLMs grandes (GPT-4, Claude, Llama, Mistral) son todos así, porque la generación cubre también muchas tareas de comprensión vía prompting.

Demo en vivo





EBIS Business
Techschool

www.ebiseducation.com

